RecognizingLicensePlates

Short project portfolio

Project goal

Build an educational license plate recognition system from scratch in Python. The project shows every stage from preprocessing and plate detection to character segmentation and OCR, with core algorithms implemented manually in NumPy.

Technologies used

- Python + NumPy for image processing and ML math.
- Pillow for image I/O and PDF demo assets.
- Tkinter for local image selection and visual pipeline inspection.
- Pytest for unit tests and sample benchmark checks.
- Gitflow-style workflow + pushed Git tags.

Main features

- Local image picker and full OCR pipeline.
- Step viewer for preprocessing, detection, normalization, segmentation, features, and OCR.
- 32x32 normalized binary character crops.
- Blended MLP, pixel-template, zoning-template, and sample-memory OCR.
- Raw character OCR without country-format forcing.

Pipeline summary

1. **Preprocess**
   grayscale, blur, CLAHE, Otsu
2. **Detect**
   Sobel, morphology, components
3. **Normalize**
   crop, Hough, rotate
4. **Segment**
   cleanup, boxes, projection
5. **Recognize**
   HOG, zoning, templates, MLP

Current verification

- Sample benchmark: 24/24
- Unit tests: 95 passed
- Archive: under 10 MB

Repository

GitHub link: https://github.com/TqLinh30/RecognizingLicensePlates

Includes source code, tests, models, samples, docs, and Gitflow history.

Seven-Step Recognition Demo

Example: data/samples/plate3.jpg. Final OCR: 18A12345.

1 Input & preprocessing

Grayscale and CLAHE enhancement.

2 Plate detection

Candidate boxes highlight the plate.

3 Plate normalization

Crop, deskew, and resize.

4 Character segmentation

Boxes isolate each character.

5 Feature extraction

Generate HOG + zoning features.

6 Classification

Blend OCR model predictions.

OCR result

18A12345

7 Final output

No country-specific formatting.

OCR result

18A12345

Repository

https://github.com/TqLinh30/RecognizingLicensePlates

Includes source code, tests, models, samples, docs, and Gitflow history.